SOLVED ASSIGNMENT

Dockerizing a Python App – Complete Step-by-Step Exercise

Aim: Create a simple Flask web application, containerize it using Docker, build a Docker image, run the application as a container, verify it, and clean up the container.

This solution follows the uploaded six-page exercise, including its exact Flask code, requirements, Dockerfile, Docker commands, port mapping, verification message, and cleanup steps.

1. PREREQUISITES

Before starting, Docker Desktop should be installed and running. Basic command-line knowledge using Terminal or Command Prompt is also required.

2. STEP 1 – CREATE THE PYTHON APPLICATION

Create a project directory and move into it:

```
mkdir python-docker-exercise
cd python-docker-exercise
```

Create a file named app.py with the following code:

```
from flask import Flask
import os

app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello from Dockerized Python! The app is running successfully."

if __name__ == "__main__":
    # Bind to 0.0.0.0 to make it accessible outside the container
    app.run(host="0.0.0.0", port=5000)
```

Create a file named requirements.txt:

```
Flask==2.3.2
Werkzeug==2.3.6
```

3. STEP 2 – CREATE THE DOCKERFILE

Create a file named exactly **Dockerfile** with no file extension in the same project directory.

```
# 1. Use the official lightweight Python image
FROM python:3.9-slim

# 2. Set the working directory inside the container
WORKDIR /app

# 3. Copy the dependencies file to the working directory
COPY requirements.txt .

# 4. Install the Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copy the rest of the application code
COPY app.py .

# 6. Expose the port the app runs on
EXPOSE 5000

# 7. Define the command to run the application
CMD ["python", "app.py"]
```

4. DOCKERFILE COMMAND EXPLANATIONS

Command	Simple Explanation
FROM python:3.9-slim	Uses the official lightweight Python 3.9 image as the base image. The slim version is smaller
WORKDIR /app	Sets /app as the working directory inside the container. Docker creates it if required.
COPY requirements.txt .	Copies requirements.txt from the host into the current container directory, /app.
RUN pip install --no-cache-dir -r requirements.txt	Installs the Python packages listed in requirements.txt during image creation. --no-cache-dir
COPY app.py .	Copies the Python application file into the container.
EXPOSE 5000	Documents that the application listens on port 5000 inside the container. It does not itself pub
CMD ["python", "app.py"]	Defines the default command that runs when the container starts. CMD runs at container sta

5. STEP 3 – BUILD THE DOCKER IMAGE

Run the following command from the project directory:

```
docker build -t my-python-app .
```

docker build: Builds a Docker image using the Dockerfile.

-t my-python-app: Gives the image the name/tag **my-python-app**.

. (dot): Specifies the current directory as the Docker build context. Docker looks there for the Dockerfile and files to copy.

6. STEP 4 – RUN THE DOCKER CONTAINER

An image is a static blueprint. Run it as a container using:

```
docker run -d -p 5050:5000 --name my-running-app my-python-app
```

Argument	Meaning
docker run	Creates and starts a container from a Docker image.
-d	Runs the container in detached/background mode and returns the terminal prompt.
-p 5050:5000	Maps host port 5050 to container port 5000 (HostPort:ContainerPort).
--name my-running-app	Assigns the container the custom name my-running-app.
my-python-app	The Docker image used to create the container.

7. STEP 5 – VERIFY THE APPLICATION

After the container starts, the Flask application is running inside the isolated Docker container.

Open a web browser and visit:

```
http://localhost:5050
```

Expected output on the web page:

```
Hello from Dockerized Python! The app is running successfully.
```

The browser uses port **5050** on the host. Docker forwards that traffic to port **5000** inside the container, where Flask is listening.

8. STEP 6 – STOP AND CLEAN UP

Stop the running container:

```
docker stop my-running-app
```

Remove the stopped container:

```
docker rm my-running-app
```

9. COMPLETE COMMAND SEQUENCE

```
mkdir python-docker-exercise
cd python-docker-exercise

# Create app.py
# Create requirements.txt
# Create Dockerfile

docker build -t my-python-app .

docker run -d -p 5050:5000 --name my-running-app my-python-app

# Open in browser:
# http://localhost:5050

docker stop my-running-app
docker rm my-running-app
```

10. PROJECT STRUCTURE

```
python-docker-exercise/
■■■ app.py
■■■ requirements.txt
■■■ Dockerfile
```

11. SIMPLE EXPLANATION OF THE COMPLETE WORKFLOW

```
Python Flask App
       ↓
requirements.txt
       ↓
Dockerfile
       ↓
docker build
       ↓
Docker Image: my-python-app
       ↓
docker run
       ↓
Container: my-running-app
       ↓
Host Port 5050 → Container Port 5000
       ↓
http://localhost:5050
```

The Python Flask application is first created. Its dependencies are listed in requirements.txt. The Dockerfile defines how to build an image. Docker then creates the image named my-python-app. The image is started as a container named my-running-app, and port 5050 on the computer is mapped to port 5000 in the container.

12. IMPORTANT CONCEPTS

Concept	Meaning
Docker Image	A reusable/static blueprint containing the application and its required environment.
Docker Container	A running instance created from a Docker image.
Dockerfile	A text file containing instructions for building a Docker image.
Flask	The Python web framework used in this exercise.
Port 5000	The port on which the Flask application listens inside the container.
Port 5050	The host port used to access the application in the browser.
-d	Runs a container in the background.
-p	Maps a host port to a container port.

13. VIVA / ORAL QUESTIONS AND ANSWERS

Q1. What is Docker?
Docker is a platform used to package and run applications in containers.

Q2. What is a Docker image?
A Docker image is a reusable blueprint used to create containers.

Q3. What is a container?
A container is a running instance of a Docker image.

Q4. What is a Dockerfile?
It is a text file containing instructions used to build a Docker image.

Q5. Why is 0.0.0.0 used in Flask?
It makes the Flask application accessible outside the container.

Q6. What does EXPOSE 5000 mean?
It documents that the application listens on port 5000 inside the container. It does not publish the port to the host by itself.

Q7. What does docker build do?
It builds a Docker image from the Dockerfile and build context.

Q8. What does -t my-python-app do?
It assigns the name/tag my-python-app to the Docker image.

Q9. What does -p 5050:5000 mean?
It maps host port 5050 to container port 5000.

Q10. What does -d mean?
It runs the container in detached/background mode.

Q11. How do you access the application?
Open http://localhost:5050 in a web browser.

Q12. How do you stop the container?
Use docker stop my-running-app.

Q13. How do you remove the container?
Use docker rm my-running-app.

14. RESULT

The Python Flask web application was successfully containerized using Docker. A Docker image named **my-python-app** was built, a container named **my-running-app** was started with port mapping 5050:5000, and the application was verified through **http://localhost:5050**. The container can then be stopped and removed using the specified Docker commands.

— END OF SOLVED ASSIGNMENT —